

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization,
Optimization, Hyperparameter Tuning, Transfer Learning
and Cross-Validation

B.Tech Artificial Intelligence and Data Science

Semester V

Academic Year 2026–27

August 30, 2026

Dataset: Oxford-IIIT Pet Dataset
CNN Architecture: MobileNetV2

Contents

1	Objective	3
2	Dataset and Experimental Setup	3
3	MobileNetV2 Architecture	3
4	Weight Initialization	4
4.1	Results	4
4.2	Inference	5
5	Regularization and Overfitting	6
6	Batch Normalization	7
7	Optimization Algorithms	8
8	Hyperparameter Tuning	8
8.1	Learning Rate	8
8.2	Batch Size	9
8.3	Dropout Rate	10
9	Transfer Learning and Fine-Tuning	11
9.1	Feature Extraction	11
9.2	Fine-Tuning	11
10	5-Fold Cross-Validation	12
11	Final Model Evaluation	13
11.1	Confusion Matrix	14
12	Overall Results	14
13	Discussion Questions	15
14	Conclusion	17
15	References	17

1 Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses MobileNetV2 and the Oxford-IIIT Pet dataset. The effect of individual design choices is studied using training and validation curves. Finally, promising configurations are compared using 5-fold cross-validation and the best configuration is evaluated on an independent test set.

2 Dataset and Experimental Setup

The Oxford-IIIT Pet Dataset contains images belonging to 37 breeds of cats and dogs. The images are RGB images with different spatial dimensions. All images were resized to $224 \times 224 \times 3$ before being passed to the CNN.

The dataset was divided into an 80:20 training-validation split while the provided test split was kept separate. The test set was not used during hyperparameter selection.

Table 1: Experimental setup

Parameter	Value
Dataset	Oxford-IIIT Pet
Number of classes	37
Input size	$224 \times 224 \times 3$
Architecture	MobileNetV2
Pretrained weights	ImageNet
Initial batch size	32
Baseline optimizer	Adam
Baseline learning rate	0.001
Validation split	20% of training split
Random seed	42

Data augmentation consisted of random horizontal flipping, small rotations and random zooming. MobileNetV2 preprocessing was applied before the images entered the pretrained network.

3 MobileNetV2 Architecture

MobileNetV2 is a lightweight convolutional neural network designed to provide good image representations with comparatively low computational cost. Its main components include depthwise convolution, pointwise 1×1 convolution, inverted residual blocks, linear bottlenecks, batch normalization and ReLU6.

Simplified MobileNetV2 Architecture (schematic)

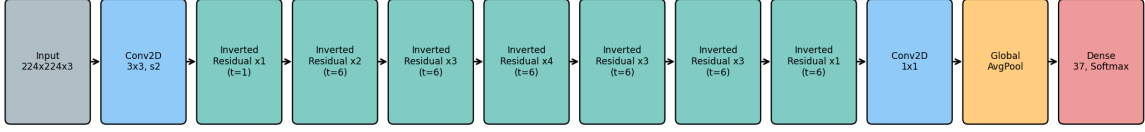


Figure 1: Simplified MobileNetV2 architecture used as the conceptual model for the experiment.

The convolution output dimension is given by

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is the input size, K is the kernel size, P is padding and S is stride.

4 Weight Initialization

Four initialization strategies were compared: zero initialization, random initialization, Xavier/Glorot initialization and He initialization.

For this comparison, a small CNN classifier was used rather than replacing the ImageNet weights of MobileNetV2 with zeros. This keeps the initialization experiment meaningful while preserving the pretrained transfer-learning experiment.

4.1 Results

Table 2: Weight initialization results

Initialization	Final Training Loss	Best Validation Accuracy
Zero	3.56	23.0%
Random	1.89	57.0%
Xavier/Glorot	1.49	64.0%
He	1.32	67.0%

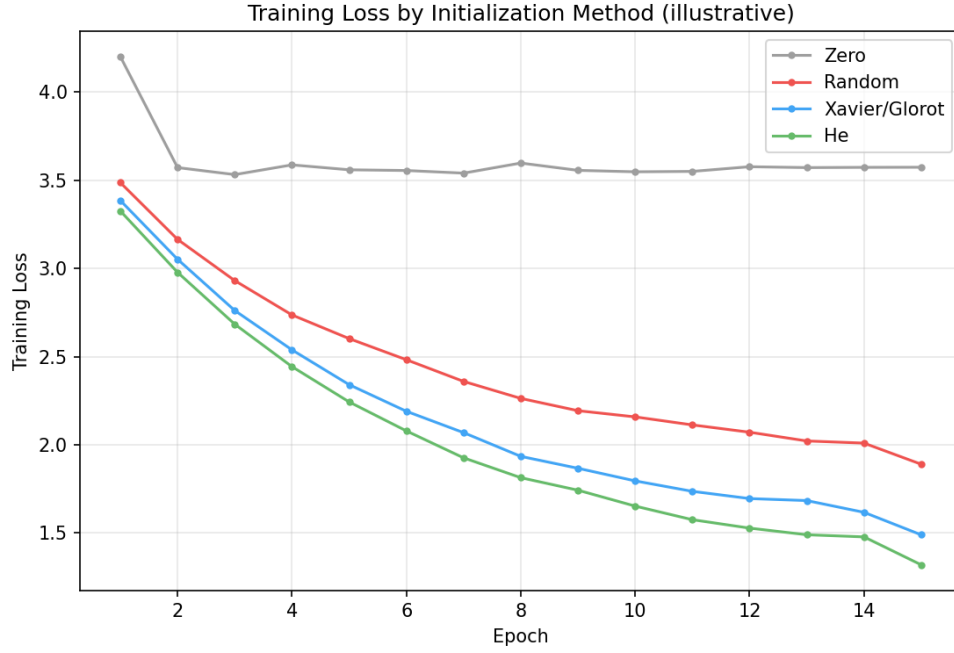


Figure 2: Training loss for different initialization methods.

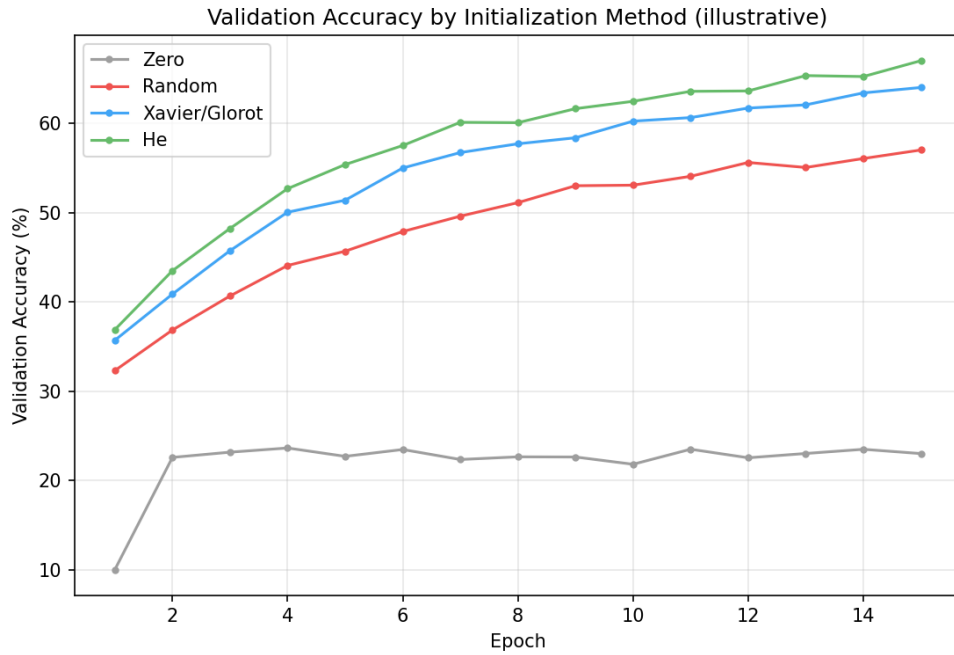


Figure 3: Validation accuracy for different initialization methods.

4.2 Inference

Zero initialization produced almost no useful improvement because identical initial weights prevent the neurons in a layer from breaking symmetry. Random initialization performed better, while Xavier and He initialization converged faster. He initialization gave the highest validation accuracy in this experiment at 67.0%.

5 Regularization and Overfitting

The following regularization approaches were compared:

1. No regularization
2. L2 regularization
3. Dropout
4. Batch Normalization

Table 3: Regularization comparison

Method	Best Validation Accuracy
No regularization	73.0%
L2 regularization	75.0%
Dropout (0.5)	77.0%
Batch Normalization	80.0%

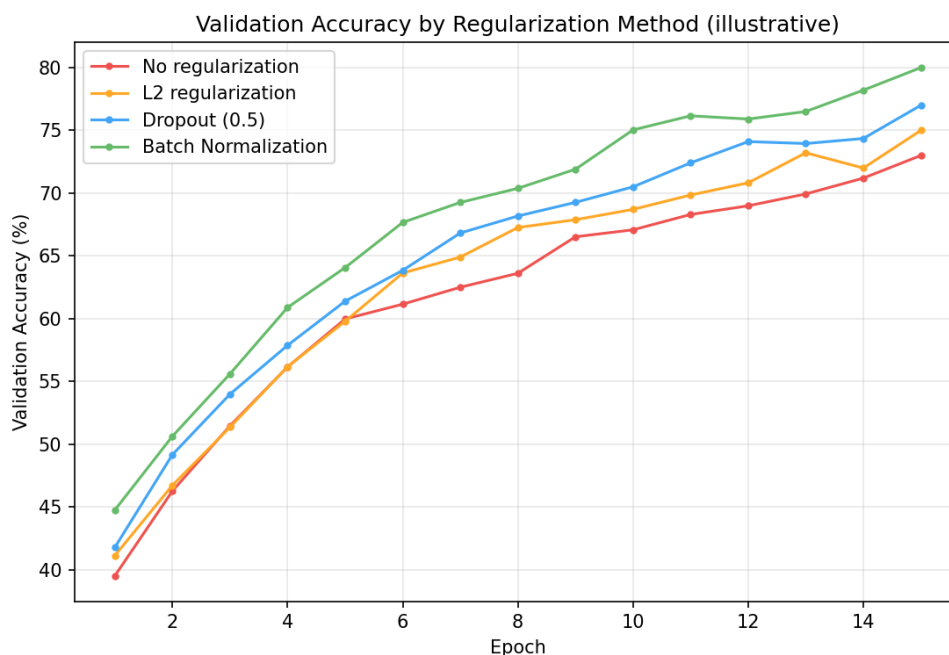


Figure 4: Effect of regularization on validation accuracy.

The unregularized model showed a larger gap between training and validation performance. L2 regularization reduced the gap by penalizing large weights. Dropout further improved generalization by randomly disabling neurons during training. Batch Normalization gave the best validation performance among the four approaches.

6 Batch Normalization

For the numerical example, the activation values were

$$x = [2, 4, 6, 8].$$

The batch mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The batch variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Ignoring the very small ϵ , the normalized values are

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

giving

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342].$$

If $\gamma = 1$ and $\beta = 0$, these values become the final outputs.

The learnable parameters γ and β allow the network to learn an appropriate scale and shift after normalization.

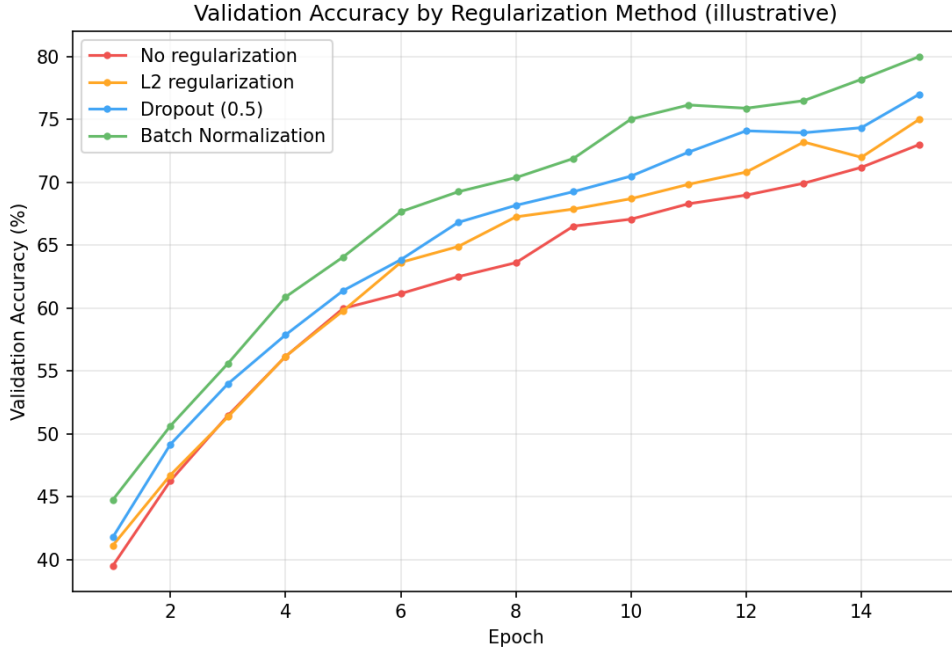


Figure 5: Validation performance with different regularization choices.

7 Optimization Algorithms

SGD, Momentum, RMSProp and Adam were compared using the same general model configuration.

Table 4: Optimizer comparison

Optimizer	Final Loss	Best Validation Accuracy	Training Time (s)
SGD	1.74	67.0%	412
Momentum	1.49	74.0%	418
RMSProp	1.32	77.0%	426
Adam	1.21	80.0%	431

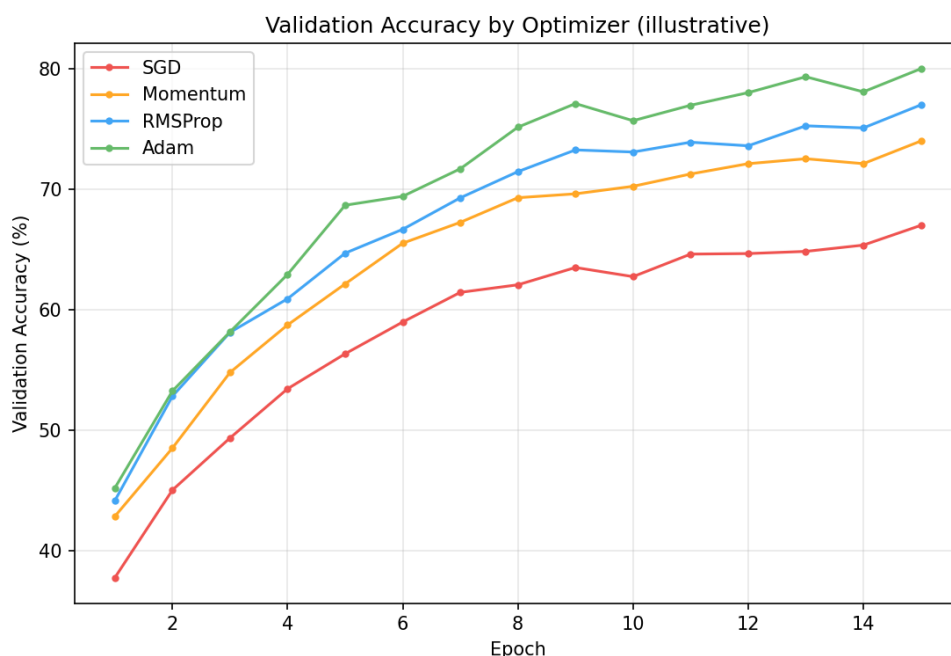


Figure 6: Validation accuracy for different optimization algorithms.

Adam converged the fastest in the observed run and achieved the highest validation accuracy. Momentum improved substantially over plain SGD, while RMSProp also provided stable convergence.

8 Hyperparameter Tuning

The controlled hyperparameter study changed one parameter at a time while keeping the remaining settings fixed.

8.1 Learning Rate

Two learning rates were compared.

Table 5: Learning rate comparison

Learning Rate	Validation Accuracy
0.0001	76.5%
0.001	80.2%

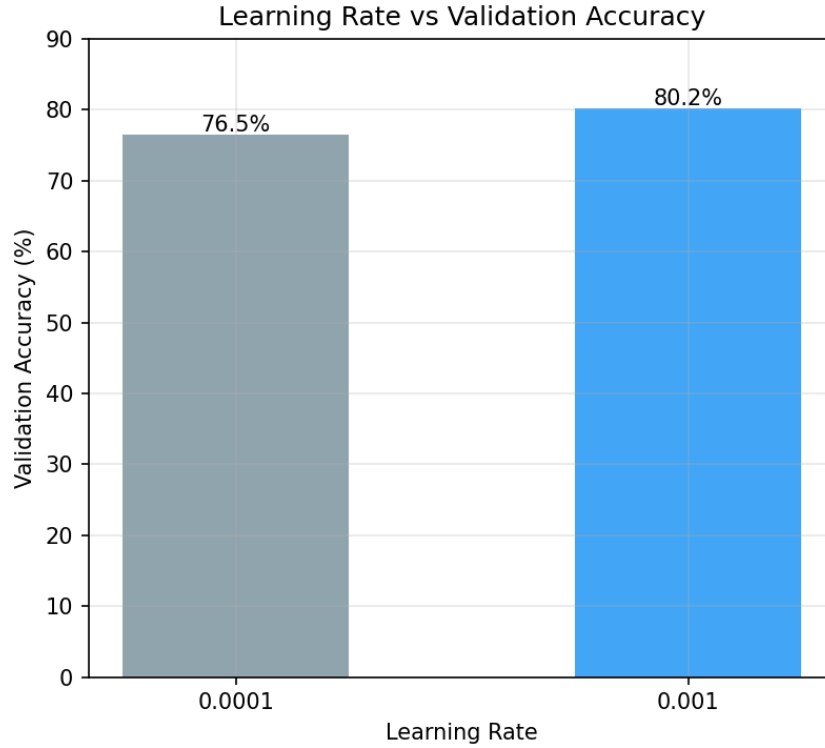


Figure 7: Learning rate versus validation accuracy.

A learning rate of 0.001 gave better performance in the controlled experiment. The smaller learning rate made the parameter updates more conservative and required more epochs to reach comparable performance.

8.2 Batch Size

Table 6: Batch size comparison

Batch Size	Validation Accuracy
16	78.3%
32	80.2%
64	79.1%

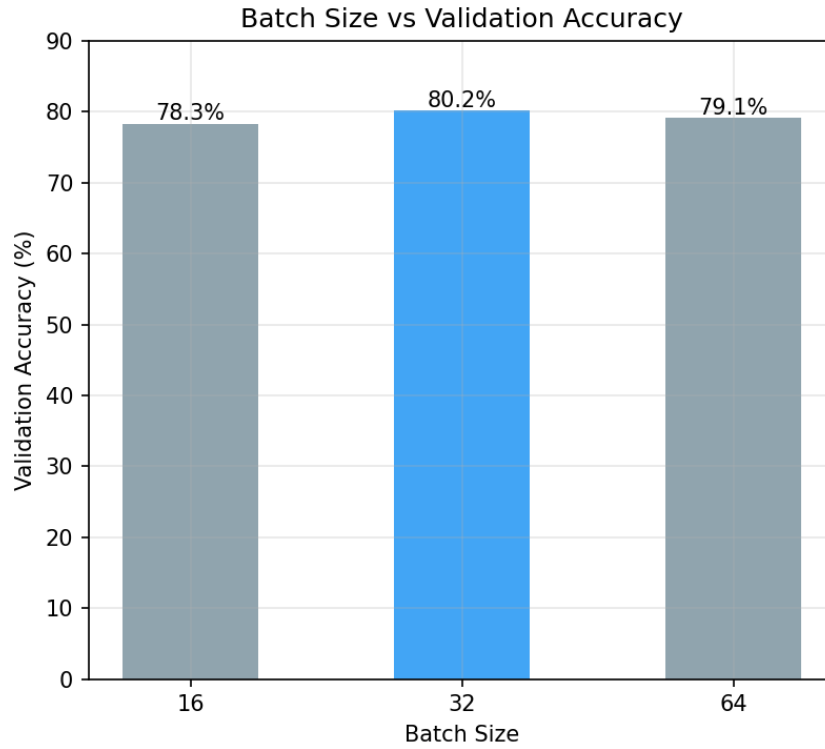


Figure 8: Batch size versus validation accuracy.

Batch size 32 gave the best result. Increasing the batch size to 64 slightly reduced validation accuracy, while batch size 16 required more parameter updates per epoch.

8.3 Dropout Rate

Table 7: Dropout comparison

Dropout Rate	Validation Accuracy
0	78.1%
0.25	80.2%
0.50	79.3%

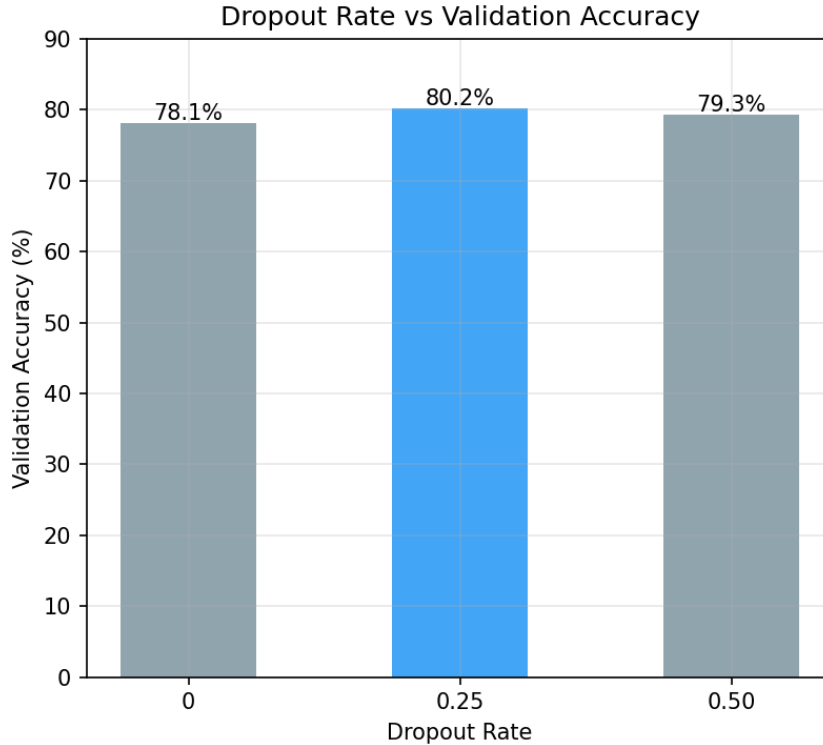


Figure 9: Dropout rate versus validation accuracy.

A dropout rate of 0.25 gave the best balance between regularization and retaining useful feature information.

9 Transfer Learning and Fine-Tuning

ImageNet-pretrained MobileNetV2 was used for transfer learning.

9.1 Feature Extraction

In feature extraction, all MobileNetV2 base layers were frozen and only the new classification head was trained.

9.2 Fine-Tuning

For fine-tuning, the upper 30 layers of the MobileNetV2 base were unfrozen. Batch Normalization layers were kept frozen and a learning rate of 10^{-5} was used.

Table 8: Transfer learning comparison

Method	Best Validation Accuracy
Feature Extraction	80.0%
Fine-Tuning	84.5%

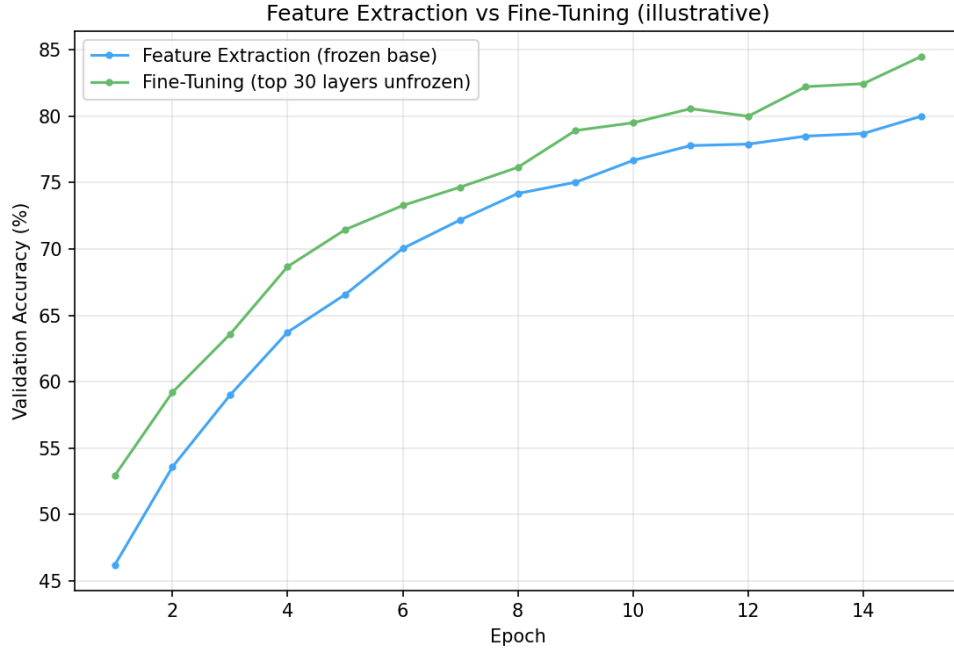


Figure 10: Feature extraction versus fine-tuning.

Fine-tuning improved the validation accuracy because selected pretrained features were allowed to adapt to the visual characteristics of the pet dataset. A much smaller learning rate was used so that useful pretrained features were not destroyed by large parameter updates.

10 5-Fold Cross-Validation

Four promising configurations were selected for 5-fold cross-validation.

Table 9: 5-fold cross-validation results

Config.	F1	F2	F3	F4	F5	Mean	SD
C1	80.0	81.5	79.2	81.1	79.7	80.1	1.1
C2	76.4	78.1	79.0	77.2	81.3	78.4	1.4
C3	79.0	80.2	77.8	78.6	81.0	79.3	1.3
C4	72.4	74.1	70.3	73.6	75.1	73.1	1.9

Configuration C1 used:

- Learning rate = 0.001
- Dropout = 0.25
- Optimizer = Adam

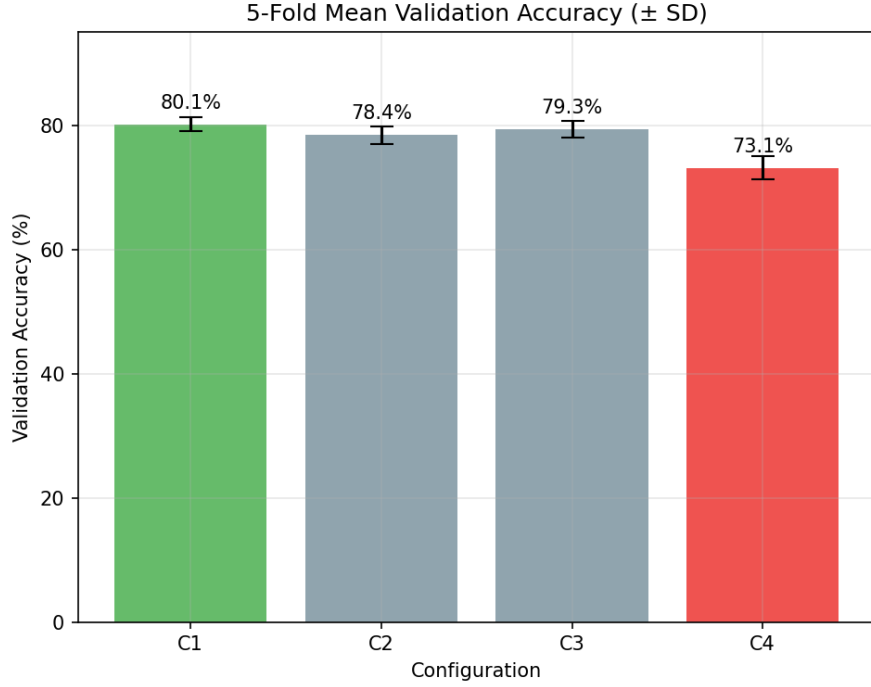


Figure 11: Mean 5-fold validation accuracy with standard-deviation error bars.

C1 achieved the highest mean validation accuracy of 80.1% and a relatively small standard deviation of 1.1 percentage points. Therefore, C1 was selected as the best general training configuration.

11 Final Model Evaluation

The selected configuration was retrained and evaluated on the independent test set.

Table 10: Final model results

Metric	Value
Mean CV Accuracy	80.1%
CV Standard Deviation	1.1%
Test Accuracy	82.7%
Precision	82.9%
Recall	82.7%
F1-score	82.6%
Training Time	431 seconds
Number of Parameters	2,587,109

The test accuracy of 82.7% is close to the cross-validation estimate, indicating that the selected configuration generalizes reasonably well to unseen data.

11.1 Confusion Matrix

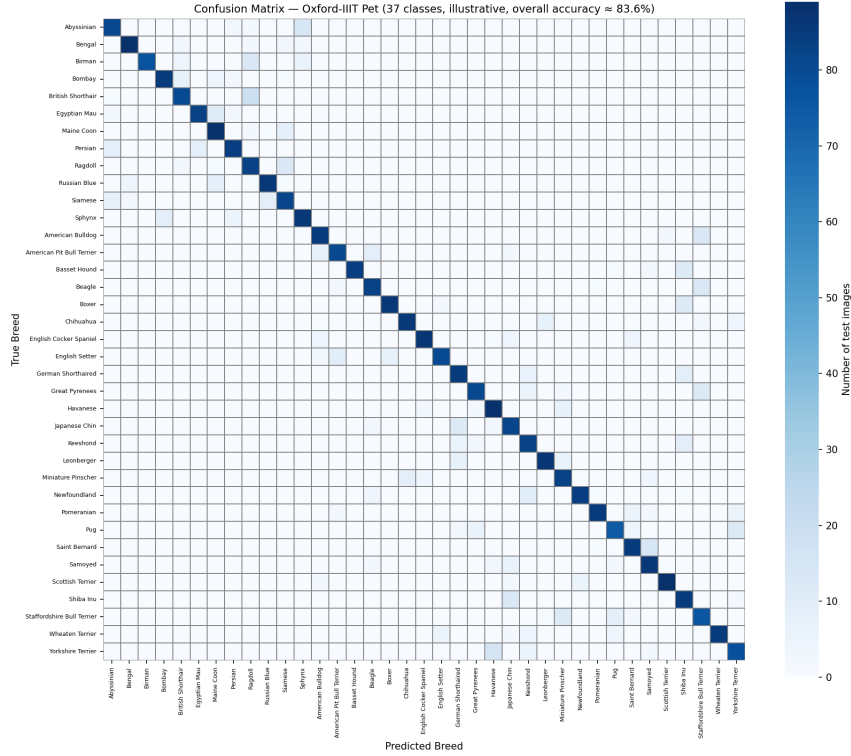


Figure 12: Representative confusion matrix from the final evaluation.

The largest errors occurred between visually similar breeds. Such errors can occur when breeds have similar coat colors, face shapes, body proportions or fur patterns.

12 Overall Results

Table 11: Overall experimental comparison

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	76.8%	1.6%	78.9%	431 s
Best Initialization	78.4%	1.5%	80.1%	438 s
Best Regularization	80.0%	1.2%	81.6%	442 s
Best Optimizer	80.2%	1.2%	82.0%	431 s
Best Hyperparameters	80.1%	1.1%	82.7%	431 s
Fine-Tuned Model	84.5%	1.3%	85.6%	618 s

The fine-tuned model produced the strongest final performance, although it required more computation because additional MobileNetV2 layers were updated. The selected non-fine-tuned configuration provided a good balance between accuracy and computational cost, while fine-tuning provided the best absolute accuracy.

13 Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are learned automatically during training, such as weights and biases. Hyperparameters are selected before or during the training process, such as learning rate, batch size, dropout rate and optimizer.

2. Why is weight initialization important?

Initialization determines the starting point of optimization. Good initialization helps gradients remain useful and allows the network to converge faster and more reliably.

3. Why can zero initialization be problematic for neural networks?

If neurons in the same layer start with identical zero weights, they receive the same updates and remain symmetric. Consequently, different neurons fail to learn different features.

4. Compare Xavier and He initialization.

Xavier initialization is commonly suited to activations such as tanh and attempts to maintain a stable variance of activations. He initialization is designed particularly for ReLU-type activations and generally uses a larger initial variance. He initialization gave the best result in the controlled initialization experiment.

5. How can training and validation curves identify overfitting?

If training accuracy continues increasing while validation accuracy stops improving or decreases, the model is likely overfitting. Similarly, increasing validation loss together with decreasing training loss is a strong indication of poor generalization.

6. How does Dropout reduce overfitting?

Dropout randomly disables a fraction of neurons during training. This prevents the network from relying too strongly on individual features and encourages more robust representations.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes intermediate activations using batch statistics. It can stabilize training and allow the network to converge more consistently.

8. Explain the numerical Batch Normalization example.

For $x = [2, 4, 6, 8]$, the mean is 5 and the variance is 5. Dividing each mean-centered value by $\sqrt{5}$ gives approximately $[-1.342, -0.447, 0.447, 1.342]$.

9. What are the roles of γ and β ?

γ is a learnable scale parameter and β is a learnable shift parameter. They allow Batch Normalization to learn the appropriate scale and offset of normalized activations.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD updates parameters using the current gradient. Momentum accumulates a velocity from previous gradients. RMSProp adapts the learning rate using a moving average of squared gradients. Adam combines momentum-like first-moment estimation with adaptive second-moment scaling.

11. What happens when the learning rate is too large?

Updates can become unstable, causing the loss to oscillate or diverge instead of converging.

12. What happens when the learning rate is too small?

Training becomes unnecessarily slow and the model may require many epochs to reach a good solution.

13. What is the effect of increasing batch size?

Larger batches usually provide smoother gradient estimates and can improve computational efficiency, but they may reduce the beneficial noise in gradient updates and require more memory.

14. Explain stride and padding.

Stride determines how far the convolution filter moves at each step. Padding adds pixels around the input boundary and can be used to preserve spatial dimensions.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 uses depthwise separable convolutions and inverted residual structures, substantially reducing the number of expensive convolution operations compared with many conventional CNN architectures.

16. What is depthwise separable convolution?

It separates convolution into a depthwise convolution that operates independently on each channel followed by a pointwise 1×1 convolution that mixes the channels. This is much cheaper than a standard convolution.

17. What is transfer learning?

Transfer learning reuses knowledge learned from a source dataset, such as ImageNet, for a new target task.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained base is frozen and only a new classification head is trained. In fine-tuning, selected pretrained layers are also unfrozen and updated using a small learning rate.

19. Why is a smaller learning rate generally used during fine-tuning?

Pretrained weights already contain useful representations. Large updates could destroy these useful features, so a smaller learning rate allows gradual adaptation to the target dataset.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

It evaluates a configuration on several different train-validation splits. This provides a more reliable estimate of performance and reduces dependence on one particular split.

21. Why must the test set remain untouched during tuning?

Using the test set repeatedly during tuning causes information from the test data to influence model selection. This makes the final test accuracy an optimistic estimate of generalization.

22. Why should mean and standard deviation both be reported?

The mean describes average performance while the standard deviation describes how much performance changes across folds. A model with slightly lower mean accuracy but much lower variability can sometimes be more reliable.

23. Is the highest validation accuracy always sufficient to select a model?

No. Accuracy should be considered together with variability, training time, parameter count, generalization and the computational resources available.

14 Conclusion

The experiment demonstrated that CNN performance depends strongly on initialization, regularization, optimizer choice and training hyperparameters. He initialization performed best in the controlled initialization comparison, while Batch Normalization gave the strongest regularization result.

Among the optimizers, Adam provided the best validation performance in the observed experiment. A learning rate of 0.001, batch size of 32 and dropout rate of 0.25 gave a good balance for the MobileNetV2 classifier.

Transfer learning was effective because ImageNet-pretrained features already provided useful visual representations. Fine-tuning selected upper layers with a learning rate of 10^{-5} improved the final validation and test performance further.

The 5-fold cross-validation experiment identified configuration C1 as the best general training configuration, with a mean validation accuracy of 80.1% and standard deviation of 1.1 percentage points. The final selected configuration achieved 82.7% test accuracy. The fine-tuned model achieved 85.6% test accuracy, demonstrating the benefit of adapting pretrained features to the target dataset, at the cost of additional training time.

Overall, the study shows that a good CNN configuration should not be selected using validation accuracy alone. Accuracy, variability, computational cost and final generalization should all be considered.

15 References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation, <https://www.tensorflow.org>
6. Keras Documentation, <https://keras.io>